

Unit 2 Notes: Back End Development

Unit 2 Notes: Back End Development

Contents

1. [Course Orientation](#)
 2. [Python Core](#)
 3. [Networking and Files](#)
 4. [Django MVT](#)
 5. [Auth and Sessions](#)
 6. [Quick Recap](#)
-

1. Course Orientation

Unit 1 built the front end: a browser that sends a request and renders whatever comes back. Unit 2 builds the other half of that conversation, the server answering it. The unit has four blocks that build on each other in order:

- **Python Core:** the language features Django leans on constantly, flexible function arguments, exception handling, and object oriented programming.
 - **Networking and Files:** what a socket actually is underneath an HTTP request, and how Python reads and writes files on disk.
 - **Django MVT:** how a Django project is organised, and how one request travels from a URL to a database row to rendered HTML.
 - **Auth and Sessions:** Django's built in login system, and what a session and a cookie actually are under the hood.
-

2. Python Core

Django is a Python framework, and it uses a specific slice of the language everywhere: flexible function signatures, disciplined error handling, and classes built around inheritance. This block covers that slice, not Python from scratch.

2.1 `*args` and `**kwargs`

A Python function can accept more arguments than the ones it names explicitly, by collecting the extra ones into a tuple or a dictionary:

```
def order_pizza(size, *toppings, **extras):  
    ...
```

```
order_pizza("L", "olive", "corn", cheese=True)
```

Parameter	Role	Value in the call above
<code>size</code>	A normal positional parameter	<code>"L"</code>
<code>*toppings</code>	Extra positional arguments, collected into a tuple	<code>("olive", "corn")</code>
<code>**extras</code>	Extra keyword arguments, collected into a dict	<code>{"cheese": True}</code>

`*` collects any positional arguments beyond the named ones into a tuple, and `**` collects any keyword arguments beyond the named ones into a dictionary. This is exactly the mechanism Django itself uses to accept a flexible, growing set of options in its own functions and class methods, so recognising the pattern here makes Django's own signatures far less mysterious later.

The same two symbols also work in reverse, at the call site rather than the definition, to unpack a collection back out into separate arguments: `order_pizza("L", *["olive", "corn"], **{"cheese": True})` sends exactly the same call as writing the arguments out by hand. This is how Django forwards a dictionary of form data or URL keyword arguments straight into a function without listing every key by name.

2.2 Exception handling flow

Python handles errors with a four part block, and each part runs at a different, well defined moment:

Block	Runs when
<code>try</code>	Always. This is the risky code being attempted.
<code>except</code>	Only if an error matched inside <code>try</code> . This is where it is handled.
<code>else</code>	Only if <code>try</code> finished with no error at all.
<code>finally</code>	Always, whether or not an error happened.

```
try:
    risky_operation()
except ValueError as e:
    handle_it(e)
else:
    only_if_no_error()
finally:
    always_runs()
```

Catch a specific error type, for example `except ValueError as e:`, and never a bare `except:`. A bare `except` catches everything, including mistakes you did not anticipate and genuinely want to see, such as a typo turning into a `NameError`, and it silently hides them.

A single `try` can carry several `except` blocks stacked in order, one per error type it needs to handle differently, and Python checks them top to bottom, running only the first one that matches. Django uses this same pattern internally: a view that cannot find a database row typically raises a specific exception, `Http404`, which Django's own error handling then catches further up and turns into a proper 404 response, rather than letting the request crash.

2.3 Classes: a blueprint and its instances

A class is a blueprint, defined once. Each time it is called, it produces an independent object, an instance, with its own copy of whatever state the blueprint describes:

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says woof"
```

`Dog(name="Rex")`, `Dog(name="Milo")` and `Dog(name="Nala")` are three separate objects built from the one `Dog` blueprint, each with its own independent `name`. `__init__` is the constructor, it runs automatically the moment an object is created and is where that object's own starting state is set up. `self` inside a method always refers to the specific instance the method was called on, which is how `self.name` can be different for Rex, Milo and Nala even though they all share the exact same `bark` method, and it is always the first parameter of every ordinary method, even though it is never passed explicitly when the method is called.

Every attribute set with `self.something` in `__init__`, such as `self.name`, belongs to that one instance and is separate for every object. This is different from an attribute written directly inside the class body, outside any method, which is shared across every instance of that class unless a specific object later overrides it.

2.4 Inheritance

A class can be defined in terms of another class, inheriting everything the parent already has and adding or changing only what is different:

```
class Animal:
    def speak(self):
        ...

class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
```

```
def speak(self):  
    return "Meow"
```

`Dog` and `Cat` are both `Animal`s, and both automatically get everything `Animal` defines, without repeating that code. Here, both children also override `speak`, method overriding, meaning a child redefines a method it inherited in order to change just that one piece of behaviour, while everything else stays shared.

When an override still needs the parent's original behaviour as part of its own, `super()` calls that parent version explicitly, for example `super().speak()` inside `Dog.speak`, rather than losing access to it entirely. This matters directly in Django, where a custom class very often overrides one method of a much larger built in class and calls `super()` to still run the framework's own original logic alongside the custom addition.

2.5 Magic (dunder) methods

A magic method, or dunder method, named with leading and trailing double underscores, is a method Python calls automatically in response to a built in operation, rather than one you call directly by name.

Method	Powers	Called as
<code>__init__</code>	Runs when an object is created	<code>obj = Dog()</code>
<code>__str__</code>	What <code>print()</code> shows for the object	<code>print(obj)</code>
<code>__len__</code>	Powers the <code>len()</code> built in	<code>len(obj)</code>
<code>__eq__</code>	Defines what <code>==</code> means for two objects	<code>a == b</code>
<code>__repr__</code>	The developer facing representation	<code>repr(obj)</code>

Without these defined, a custom object prints as a fairly meaningless memory address and cannot be compared with `==` in any way that reflects its actual data. Defining `__str__` and `__eq__` is what makes a custom class feel like it behaves as naturally as Python's own built in types.

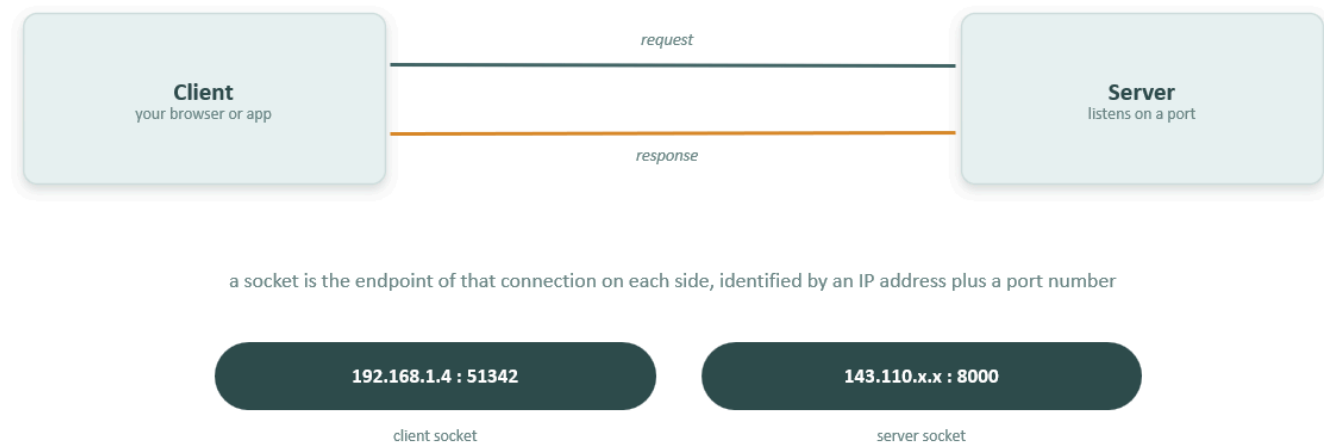
`__str__` and `__repr__` are easy to mix up: `__str__` is the readable version meant for a person, what `print()` shows, while `__repr__` is the unambiguous, developer facing version meant for debugging, ideally precise enough to recreate the object. If only one is defined, Python falls back to `__repr__` wherever `__str__` would otherwise be used, which is why `__repr__` is generally worth defining even when `__str__` is also present.

3. Networking and Files

3.1 Recap: client, server, and what a socket needs

Unit 1 covered the request and response at the level of a browser and a URL. Underneath that exchange, both sides communicate through a socket: the endpoint of a connection on each side,

identified by an IP address plus a port number.



Every request travels from a specific client socket to a specific server socket, and the response travels back along the same pairing. A port is simply which of many possible doors on that machine the connection is using, since one server machine can run several services at once, each listening on its own port. Port 8000 in the diagram is Django's own default development port, `python manage.py runserver` listens there unless told otherwise, and port 80 is the default for ordinary, unencrypted HTTP traffic in production.

3.2 Opening a socket in Python

Python's `socket` module gives direct access to this layer, the exact layer that HTTP, and Django underneath it, are built on top of:

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("example.com", 80))
```

Call	Role
<code>socket()</code>	Create an endpoint
<code>connect()</code>	Reach out to a server
<code>send()</code> / <code>recv()</code>	Exchange bytes
<code>close()</code>	Release the connection

HTTP is, underneath all of Django's abstraction, just a specific text protocol spoken over exactly this kind of socket. A request is a block of text sent with `send()`, a response is a block of text read back with `recv()`, and everything Django, or any web framework, does on top of this is convenience built over these four calls. `AF_INET` says the address is a normal IP address, and `SOCK_STREAM` says the connection behaves as TCP, a reliable, ordered stream of bytes, which is the specific guarantee HTTP is built to assume.

3.3 HTTP methods, at a glance

An HTTP method states the intent of a request, what the client is trying to do to a resource, not just which resource it is asking about:

Method	Intent
GET	Read a resource
POST	Create a resource
PUT	Replace a resource
DELETE	Remove a resource

```
GET /api/orders/42 HTTP/1.1
```

```
Host: example.com
```

```
Authorization: Bearer <token>
```

```
200 OK -> a JSON body with the order details
```

This maps directly onto the request anatomy from Unit 1, method, path, headers, and a response with a status code and a body. Django's views, covered in section 4, are essentially functions written to answer exactly this shape of request, one method and one path at a time, and a single view function commonly checks `request.method` to decide whether it is handling a `GET`, showing a form or a page, or a `POST`, acting on submitted data. `GET` and `PUT` are also expected to be idempotent, repeating them has the same effect as doing it once, which is part of why forms that create something, an order or a comment, are sent as `POST` rather than `GET`.

3.4 Reading and writing files

Python opens a file in one of several modes, and the mode chosen decides both what is allowed and what happens to any existing content:

Mode	Behaviour
<code>r</code>	Read. The file must already exist.
<code>w</code>	Write. Overwrites the file completely.
<code>a</code>	Append. Adds to the end of the file.
<code>r+</code>	Read and write, no truncation of existing content.

```
with open("data.txt", "r") as f:  
    text = f.read()
```

The `with` block is the standard pattern for opening a file: the file closes itself automatically the moment the block ends, even if an error happens partway through, which avoids the class of bugs where a file is

accidentally left open. Adding `b` to any of these modes, for example `rb` or `wb`, switches to binary mode, reading or writing raw bytes instead of decoded text, which matters for anything that is not plain text, such as an image or a PDF.

4. Django MVT

4.1 The Model, View, Template cycle

Django organises an application around three kinds of file, each with one clear job, and the pattern is called MVT:

Part	File	Job
Model	<code>models.py</code>	The data
View	<code>views.py</code>	The logic
Template	<code>templates/</code>	The HTML

`urls.py` routes an incoming request to a view, the view asks a model for data, and the view fills that data into a template to produce the final HTML response. Keeping these three concerns in three different kinds of file, rather than mixed together, is the whole reason MVT scales to a large application: a designer can edit a template without touching Python, and the data layer can change without rewriting every page that displays it.

MVT is Django's own name for a pattern most other frameworks call MVC, model, view, controller. The mapping is not one to one by name: what Django calls a view is closer to what MVC calls the controller, the logic that decides what happens, while the actual rendering step, what MVC calls the view, is Django's template.

4.2 Project vs app

A Django project and a Django app are not the same thing, and confusing them is a common early mistake:

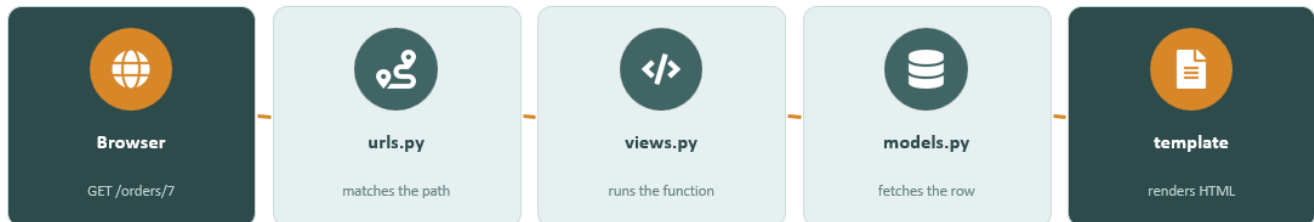
myproject/	myapp/
<code>settings.py</code>	<code>models.py</code>
<code>urls.py</code>	<code>views.py</code>
<code>wsgi.py</code>	<code>urls.py</code>
	<code>templates/</code>

The project is one per site: it holds the global settings and the root URL table that every request first arrives at. An app is one per feature, for example orders, accounts, or payments, and a single project can, and usually does, hold many apps. This split is what makes an app reusable across different projects, since an app does not hardcode anything about the specific project it happens to be plugged into.

An app only actually takes effect once it is listed in `INSTALLED_APPS` inside the project's `settings.py`. Creating the app's folder with `startapp` is not enough on its own, Django will not pick up its models or templates until that registration step is done, which is a common source of a model or template silently appearing not to exist.

4.3 Anatomy of one request

Putting MVT and the project/app split together, a single request makes a fixed round trip through five stops:



the response makes the same trip back, template → view → browser

The response then makes the same trip back in reverse, template to view to browser. Every Django request, however complex the page, is fundamentally this same five step trip: a URL is matched, a view function runs, that function may ask a model for data, and a template turns the result into HTML to send back. A view does not have to render a template at all, it can just as easily return a redirect to a different URL, or return data directly as JSON instead of HTML, which is the same basic pattern an API endpoint built with Django REST Framework, covered later in the course, is built on.

4.4 From a model to a database table

A model is a Python class, but Django turns it into a real SQL table through a two step, version controlled process:

```
class Order(models.Model):  
    ...
```

Step	Command	What happens
1	Define the model	Write the class in <code>models.py</code>
2	<code>makemigrations</code>	Write the change as a migration file
3	<code>migrate</code>	Apply that file to the database
Result	<code>orders_order</code>	The real table now exists

Migrations are version control for the database schema: every change to a model is captured as its own file, in order, and that file is committed and shared with the team through Git exactly like any other source file, covered in Unit 1. This is what lets a whole team keep their local databases in sync with the

model definitions as those definitions evolve. `makemigrations` only writes the migration file, it does not touch the database, `migrate` is the separate step that actually applies any pending migration files, which is why a change to `models.py` that is never followed by both commands never actually reaches the real table.

4.5 Templating basics

A Django template is HTML with two small additions: a way to print a value, and a way to add simple logic, without ever running arbitrary Python inside the page itself.

```
<h1>Hi {{ user.name }}</h1>

{% if orders %}
    <p>{{ orders|length }} orders</p>
{% endif %}
```

Syntax	Purpose
<code>{{ variable }}</code>	Prints a value that the view passed into the template's context
<code>{% tag %}</code>	Logic: <code>if</code> , <code>for</code> , <code>block</code> , and so on

The deliberate restriction here matters: a template can display and lightly branch on data, but it can never execute raw Python, which keeps the separation between logic (in `views.py`) and presentation (in the template) enforced by the framework itself, rather than left as a convention someone can break.

Two further pieces come up in almost every real template. A filter, the `|length` after `orders` above, transforms a value inline, other common ones include `|date` and `|default`. Template inheritance lets one base template define the shared page layout once, with `{% block content %}` marking the parts a specific page is allowed to fill in, and every other template then opens with `{% extends "base.html" %}` instead of repeating the surrounding HTML on every single page.

5. Auth and Sessions

5.1 `django.contrib.auth`, out of the box

Django ships with a complete authentication system already built, `django.contrib.auth`, so a login flow does not need to be written from scratch:

```
from django.contrib.auth import authenticate, login

user = authenticate(
    request, username=u, password=p
)
```

```
if user is not None:
    login(request, user)
```

`authenticate()` checks a username and password against the database and returns the matching user, or `None` if they do not match. `login()` then attaches that user to the current session, covered in section 5.3. The built in `User` model already carries the fields a real login system needs: `username`, a `password` that is always hashed, never stored as plain text, and the three boolean flags `is_active`, `is_staff` and `is_superuser` that control what the account is allowed to do. A new user is created with `User.objects.create_user(...)`, never `User(...)` directly, since `create_user` is what actually runs the password through Django's hashing before saving it. Logging out is the direct counterpart, `logout(request)` clears the session so the next request from that browser is anonymous again.

5.2 Permissions and decorators

Once a login system exists, individual views need to be protected, restricted to logged in users, or further restricted to users holding a specific permission. Django does this with decorators placed directly above a view function, plus two checks usable inside a view or a template:

Tool	Effect	Where
<code>@login_required</code>	Redirects anonymous users to the login page	<code>views.py</code>
<code>@permission_required</code>	Blocks the view unless the user holds that permission	<code>views.py</code>
<code>request.user.is_authenticated</code>	<code>True</code> once a user is logged in	templates and views
<code>request.user.has_perm(...)</code>	Checks one specific permission in code	<code>views.py</code>

A decorator wraps a view function without editing its body, so protecting a view is usually a one line addition directly above its definition, rather than a block of conditional logic written inside every protected view by hand. Individual permissions are normally bundled into a Group, for example "editors" or "support staff", rather than assigned to each user one by one, so `@permission_required` in practice is usually checking membership in one of those groups.

5.3 What a session actually is

A session is how Django recognises the same browser across multiple separate requests, and understanding it means separating what lives in the browser from what lives on the server:

Browser	Server
cookie: sessionId = 8k3j...f91	--> django_session table: key · data · expiry

Only a session ID travels in the cookie sent to the browser, the real session data itself stays on the server, stored in the `django_session` table. This is why `login()` just works: Django writes one row into that session table, sets one cookie on the response containing nothing but the key to that row, and

every later request from that same browser carries the cookie back, letting Django look up the matching row and know which user is making the request, without the password or any sensitive data ever needing to travel back and forth again.

A session row also carries an expiry, `SESSION_COOKIE_AGE` in settings controls how long, by default, a session stays valid, and past that point the cookie no longer matches any live row and the user is treated as logged out again. Sessions are also what makes CSRF protection possible: Django ties a one time token to the session and expects every `POST` form to send it back, which is how it tells a genuine submission from your own page apart from one forged by another site.

6. Quick Recap

- `*args` / `**kwargs`: collect extra positional and keyword arguments.
- `try` / `except` / `finally`: handle errors without crashing the program.
- **Classes and inheritance**: a blueprint, its instances, and children that override methods.
- **Sockets and HTTP**: the request and response exchange underneath every web call.
- **MVT**: model holds data, view holds logic, template holds HTML.
- **Migrations**: the version controlled path from a model to a real database table.
- **Auth and sessions**: `login()` sets one cookie, the real data stays server side, a protected view now guards real users, with no third party package required.